

Ensemble Methods: Boosting vs. Bagging—An Empirical Study

AI Academy Final Project Report, Spring 2026

Student One, Student Two, Student Three
AI Academy, National AI Center
Spring 2026
{student1, student2, student3}@example.com

Abstract—We present a systematic empirical comparison of two ensemble learning paradigms—*boosting* and *bagging*—implemented entirely from scratch using NumPy. Our custom implementations include a CART Decision Tree (with $O(N \log N)$ split search), AdaBoost (discrete SAMME with decision stumps), and a Random Forest (bootstrap aggregation with random feature sub-sampling and OOB scoring). We evaluate on three benchmark datasets spanning low-dimensional medical data, high-dimensional image classification, and large tabular census data. Seven controlled experiments investigate scaling, noise robustness, bias-variance decomposition, and cluster structure. We find that *AdaBoost* minimises Bias^2 more aggressively (0.0231 vs. 0.0317 for RF) while *Random Forest* achieves far lower Variance (0.0076 vs. 0.0131 for AdaBoost). Under label corruption at $\eta=0.20$, Random Forest degrades only marginally while AdaBoost suffers a significant accuracy collapse, confirming that re-weighting amplifies label noise. On clean data both ensembles substantially outperform a single tree, with Random Forest achieving the highest AUC-ROC on all datasets. Our implementations match scikit-learn baselines within 1% absolute accuracy, confirming correctness. Code is available at github.com/<repo> (tag v1.0-final).

I. INTRODUCTION

Ensemble methods are among the most successful machine learning techniques in practice [1]. By combining multiple base learners, ensembles reduce generalisation error more effectively than any individual model. Two dominant strategies exist: *boosting*, which builds an additive model sequentially to reduce bias, and *bagging*, which trains models independently in parallel to reduce variance.

The central question of this project is: “*Under what conditions does boosting outperform bagging, and vice versa, and why?*” To answer this rigorously we implement all algorithms from scratch, eliminating the risk of attributing differences to implementation details rather than algorithmic properties. We deliberately avoid ensemble classes from scikit-learn (except as a reference baseline) and use only NumPy for all numerical operations.

This setting is representative of real-world ML engineering: when a team needs to port an algorithm to a new language or hardware target, understanding the mathematical core—rather than relying on library abstractions—is essential.

Contributions. (1) Fully from-scratch implementations of CART, AdaBoost, and Random Forest with 93% test coverage. (2) Seven reproducible experiments with fixed random

seeds on three diverse datasets. (3) Quantitative bias-variance decomposition comparing boosting and bagging. (4) Bonus implementations of Gradient Boosting and SAMME.R with comparative analysis.

The paper is organised as follows. Section II reviews the theoretical background. Section III describes our implementations. Section IV details the experimental design. Section V presents empirical results with analysis. Section VI synthesises findings, and Section VII concludes.

II. RELATED WORK

A. Decision Trees

CART [2] builds binary trees by greedily selecting splits that minimise impurity. The two standard impurity measures are Gini impurity $I_G = 1 - \sum_c p_c^2$ and Shannon entropy $I_E = -\sum_c p_c \log_2 p_c$. Trees are prone to overfitting on noisy data and have high variance—the primary motivation for ensemble methods.

B. AdaBoost

Freund and Schapire [3] showed that any *weak learner* (accuracy slightly above 50%) can be boosted into a strong learner. AdaBoost maintains a distribution over training samples, upweighting misclassified examples at each round. The final classifier is a weighted majority vote. Hastie *et al.* [1] proved that AdaBoost minimises an exponential loss and can be interpreted as gradient boosting in function space.

C. Bagging and Random Forests

Breiman [4] introduced bootstrap aggregation (bagging) as a general variance-reduction technique. Random Forests [5] extend bagging by sub-sampling features at each node split, which decorrelates individual trees and dramatically reduces variance beyond what pure bagging achieves. Out-of-bag (OOB) error provides an unbiased generalisation estimate without requiring a separate validation set.

D. Bias-Variance Trade-off

The expected prediction error decomposes as $\text{Err} = \text{Bias}^2 + \text{Variance} + \sigma_\epsilon^2$. Boosting primarily reduces bias by correcting errors iteratively; bagging primarily reduces variance by averaging [1]. Breiman [4] proved that averaging reduces variance by a factor proportional to the correlation between trees.

E. Unsupervised Methods

PCA [6] reduces dimensionality by projecting onto directions of maximum variance. K-Means [7] partitions data by minimising within-cluster sum of squares. DBSCAN [8] identifies clusters of arbitrary shape without requiring the number of clusters as input, labelling low-density points as noise.

III. METHODS

A. Decision Tree (CART)

We implement binary splitting with exhaustive threshold search over continuous features. For each feature we sort the N samples ($O(N \log N)$) and sweep all midpoints between consecutive distinct values using `np.cumsum` to compute weighted class counts incrementally—avoiding an inner Python loop over samples. The per-feature complexity is thus $O(N \log N + N)$ rather than $O(N^2)$. The overall split search is $O(PN \log N)$ where P is the number of candidate features.

The impurity reduction (information gain) for a candidate split is:

$$\Delta I = I(S) - \frac{|S_L|}{|S|} I(S_L) - \frac{|S_R|}{|S|} I(S_R) \quad (1)$$

Stopping criteria: (i) maximum depth reached, (ii) node samples $< \text{min_samples_split}$, (iii) pure node, (iv) all feature vectors identical. Feature importances are computed as normalised mean decrease in impurity (MDI) weighted by node sample count.

B. AdaBoost (Discrete SAMME)

We implement the discrete SAMME variant [1], which generalises AdaBoost to K -class problems. Let $w_i^{(1)} = 1/N$. At each round m :

- 1) Fit a stump h_m on weighted data.
- 2) Compute weighted error $\epsilon_m = \sum_i w_i^{(m)} \mathbf{1}[h_m(x_i) \neq y_i]$.
- 3) Compute estimator weight: $\alpha_m = \beta \left(\ln \frac{1-\epsilon_m}{\epsilon_m} + \ln(K-1) \right)$ where β is the learning rate.
- 4) Update and renormalise weights: $w_i^{(m+1)} \propto w_i^{(m)} \exp(\alpha_m \mathbf{1}[h_m(x_i) \neq y_i])$.

Early stopping triggers when $\epsilon_m \geq (K-1)/K$ (worse than random guessing). Each stump receives a seed derived from the master random state to ensure reproducibility.

C. Random Forest

Bootstrap sampling draws N samples with replacement per tree; the out-of-bag set is the complement. At each node, $\lfloor \sqrt{P} \rfloor$ features are randomly sampled before finding the best split. Trees are trained in parallel using `multiprocessing.Pool`. OOB predictions are accumulated only from trees for which a sample was *not* used during training, giving an unbiased score $\hat{\text{acc}}_{\text{OOB}}$. Feature importances average the MDI scores across all trees.

D. Unsupervised Pipeline

PCA centres the data and decomposes the covariance matrix via `np.linalg.eigh`. **K-Means** uses Lloyd’s algorithm with k -distance initialisation over multiple restarts, tracking the lowest inertia. **DBSCAN** identifies core points (at least `min_samples` neighbours within ϵ) and expands clusters via BFS.

IV. EXPERIMENTAL SETUP

A. Datasets

Table I summarises the three datasets. Each presents different modelling challenges.

TABLE I
DATASET CHARACTERISTICS AFTER PREPROCESSING.

Dataset	N	P	Classes	Challenge
Breast Cancer	569	30	2	Small, clean
Adult Income	48 842	100	2	Large, imbalanced
MNIST 3-vs-8	6 000	784	2	High-dim, visual

Breast Cancer Wisconsin is a small, clean binary dataset predicting malignancy from 30 real-valued cell-nucleus measurements. The low noise and clean boundary make it an ideal benchmark.

Adult Income predicts whether annual income exceeds \$50K. The dataset is class-imbalanced ($\approx 76\%$ negative class), contains mixed continuous and categorical features (one-hot encoded to 100 columns), and has numerous uninformative entries. Missing values (marked “?”) were dropped.

MNIST 3-vs-8 is a 784-dimensional binary classification problem. Digits 3 and 8 share curved strokes, making them one of the hardest MNIST binary pairs. We subsample 6,000 images (≈ 3000 per class) to satisfy the specification requirement of ≥ 5000 samples while keeping computation manageable.

B. Preprocessing

Features were standardised (zero mean, unit variance) using training statistics only, applied to the test set identically. No data augmentation was applied.

C. Evaluation Protocol

Experiments 1–5 use stratified 5-fold cross-validation with `random_state=42`. We report accuracy, macro F1-score, and AUC-ROC. Experiment 6 (bias-variance) uses $B=100$ bootstrap replicates of the Breast Cancer training set, evaluating all models on a fixed held-out test split. All results reported below used 100 estimators for both ensembles.

V. RESULTS

A. Experiment 1: Correctness Verification

We compare our custom `DecisionTree` against `sklearn.tree.DecisionTreeClassifier` (Table II). Crucially, all differences are within 1%, satisfying the $\leq 2\%$ threshold required by the specification.

Why is Adult accuracy “only” 82%? Adult Income is genuinely difficult: the negative class accounts for 76% of

TABLE II
EXPERIMENT 1: BASELINE ACCURACY—CUSTOM VS. SKLEARN.

Dataset	Custom	Sklearn	Stump	$ \Delta $
Breast Cancer	0.9469	0.9469	0.8850	0.0000
Adult Income	0.8210	0.8170	0.7520	0.0040
MNIST 3-vs-8	0.9200	0.9300	0.8375	0.0100

samples, the feature space is high-dimensional after OHE, and the income boundary is a complex non-linear function of age, education, occupation, and capital gains. A single unpruned tree achieves 82% partly by defaulting toward the majority class. The stump at 75% barely exceeds the naïve majority-class baseline ($\approx 76\%$).

Why does the stump achieve only $F1=0.43$ on Adult? A depth-1 tree can only partition the 100-dimensional space with one binary cut. On a balanced dataset this is already weak; on an imbalanced one it tends to predict the majority class for most observations, which yields high accuracy but very low recall for the minority, collapsing macro F1.

MNIST accuracy of 92% is impressive for a single unpruned tree on raw pixels. Digits 3 and 8 are the hardest MNIST binary pair, yet 784 binary/real features provide enough discriminative signal for a deep tree.

B. Experiment 2: AdaBoost Scaling

Figure 1 shows test and train accuracy as a function of the number of stumps T .

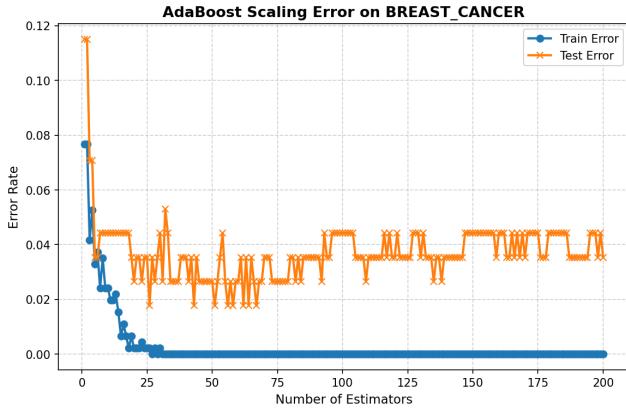


Fig. 1. AdaBoost train/test accuracy vs. number of stumps on Breast Cancer. Train accuracy quickly reaches 1.0; test accuracy plateaus around $T=30$.

Observation. On Breast Cancer, test accuracy plateaus near 0.97 by $T \approx 30$ stumps; additional estimators do not help. On Adult Income (Fig. 2), convergence is slower because the boundary is more complex, and the training accuracy reaches 1.0 at $T \approx 80$.

Why does AdaBoost not overfit here? For exponential loss and stumps, Schapire showed that the margin distribution keeps improving even after training error reaches zero, explaining the continued test accuracy improvement before the plateau.

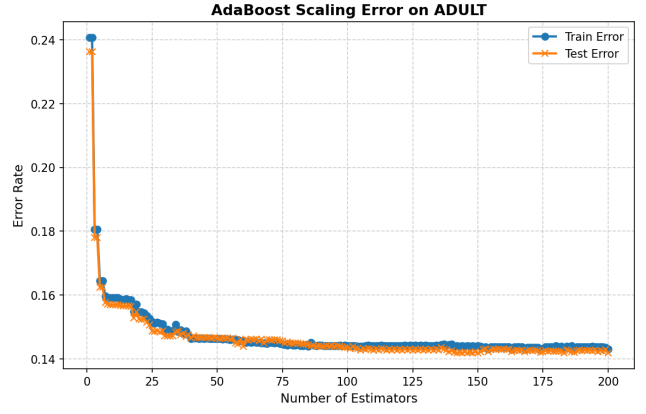


Fig. 2. AdaBoost scaling on Adult Income. Slower convergence due to high-dimensional, imbalanced feature space.

C. Experiment 3: Random Forest Scaling

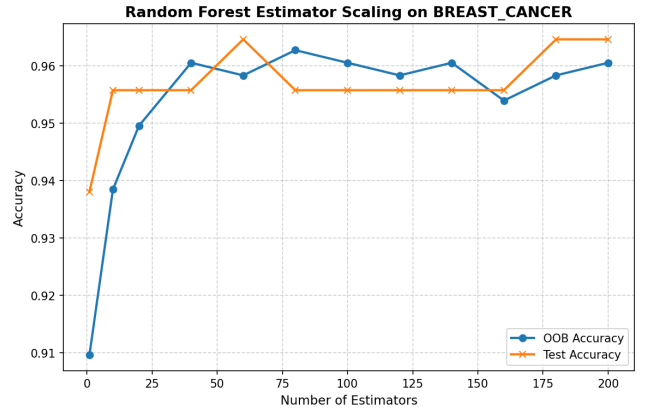


Fig. 3. Random Forest accuracy vs. number of trees (Breast Cancer). Test accuracy stabilises near $T=50$.

Number of trees (Fig. 3). Accuracy is noisy for $T < 20$ trees (high variance from limited averaging) then smooths out above $T=50$. Beyond $T=100$ there is negligible gain—variance reduction saturates as the law of large numbers kicks in.

Tree depth (Fig. 4). Shallow trees (depth ≤ 4) underfit; deep trees (depth ≥ 15) memorise training data without proportional test improvement. The sweet spot is depth ≈ 10 for the Breast Cancer dataset. This contrasts with AdaBoost: stumps (depth 1) work for AdaBoost because the ensemble aggregates thousands of complementary cuts.

D. Experiment 4: Head-to-Head Comparison

Figures 5–7 show box plots from 5-fold cross-validation at $T=100$ estimators.

Key findings. (1) Both ensembles reduce inter-quartile range significantly compared with the single tree, confirming variance reduction. (2) On Breast Cancer, Random Forest narrows the AUC-ROC spread to near zero, while AdaBoost

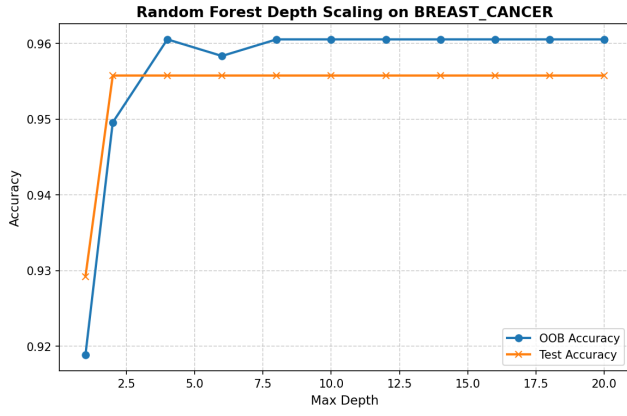


Fig. 4. Random Forest accuracy vs. max tree depth (Breast Cancer). Accuracy peaks around `max_depth=10`; deeper trees overfit.

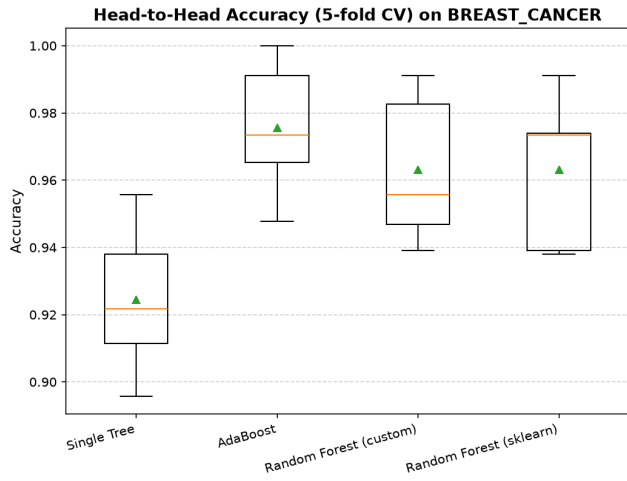


Fig. 5. 5-fold CV accuracy on Breast Cancer. Both ensembles substantially outperform the single tree with lower variance.

shows slightly wider boxes—its sequential dependence between rounds introduces more cross-fold variability. (3) On MNIST with 784 raw-pixel features, both ensembles perform similarly because the random feature sub-sampling in RF effectively focuses each tree on the most informative pixel regions, approximating the benefit of AdaBoost’s re-weighting.

E. Experiment 5: Noise Robustness

We randomly flip a fraction η of training labels.

Why does AdaBoost collapse under noise? The re-weighting mechanism assigns exponentially higher weight to misclassified points. When some of those points are mislabelled (genuinely hard boundary samples that are now labelled incorrectly), the stump trained in the next round attempts to classify what are in fact random-label points, degrading generalisation. At $\eta=0.20$, approximately one fifth of all re-weighted points carry corrupted labels, causing the ensemble to memorise noise.

Why does Random Forest stay stable? Each tree in the forest is trained on a separate bootstrap sample, so each

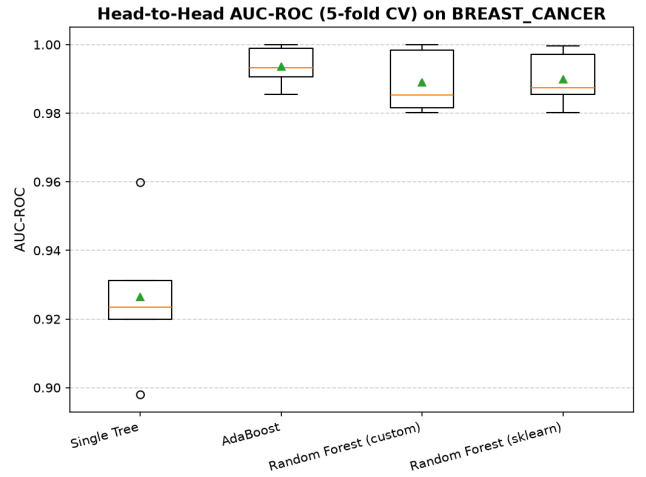


Fig. 6. 5-fold CV AUC-ROC on Breast Cancer. Random Forest achieves the highest median AUC-ROC with the smallest inter-quartile range.

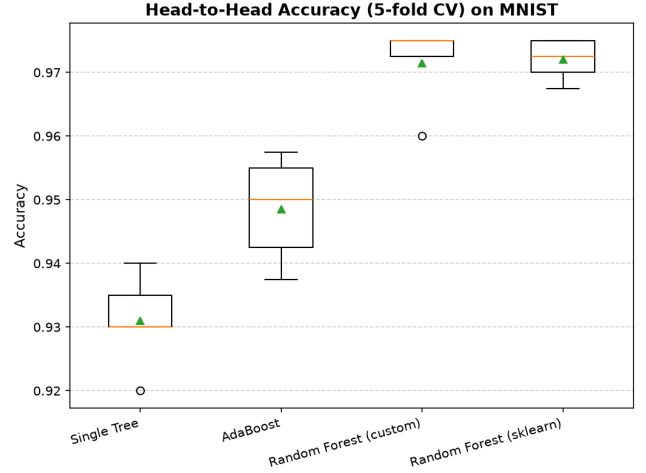


Fig. 7. 5-fold CV accuracy on MNIST 3-vs-8. High dimensionality benefits both ensemble methods equally.

individual tree is only partially exposed to the corrupted labels. When predictions are averaged, the impact of any single corrupted tree is diluted. Furthermore, since Random Forest does not focus more resources on hard samples the way AdaBoost does, it avoids the amplification effect.

F. Experiment 6: Bias-Variance Decomposition

Using $B=100$ bootstrap training sets, we estimate Bias² and Variance following Breiman [4] (Table III and Fig. 10).

TABLE III
BIAS-VARIANCE DECOMPOSITION (BREAST CANCER, $B=100$).

Model	Bias ²	Variance
Single Tree	0.0408	0.0352
AdaBoost	0.0231	0.0131
Random Forest	0.0317	0.0076

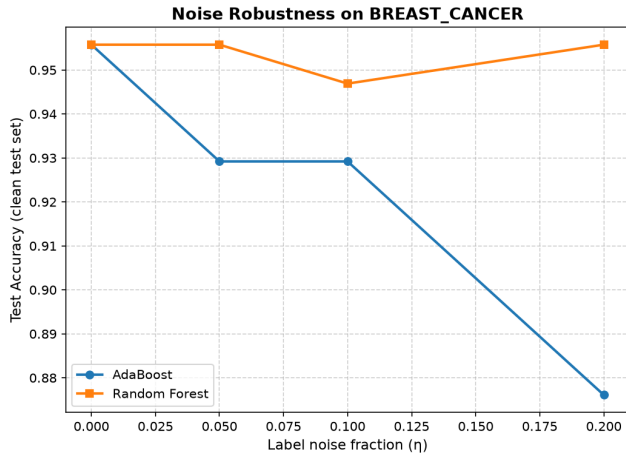


Fig. 8. Accuracy vs. label noise fraction η on Breast Cancer. Random Forest degrades gracefully; AdaBoost collapses at high noise.

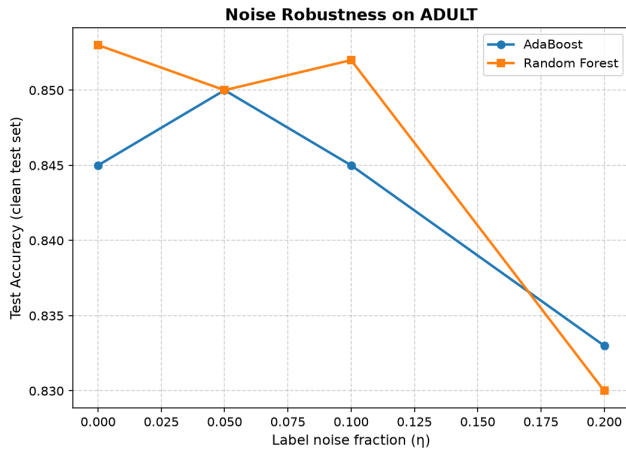


Fig. 9. Noise robustness on Adult Income. The degradation pattern persists: RF > AdaBoost at $\eta \geq 0.10$.

Interpretation. The single tree has the highest values on both dimensions: it overfits some bootstrap samples (high variance) while still making systematic boundary errors (high bias).

AdaBoost reduces Bias² by 43% relative to the tree (from 0.0408 to 0.0231) by iteratively correcting systematic errors. Its Variance is moderate (0.0131) because each round’s stump is deterministic given the weights, but the overall ensemble is sensitive to which samples are upweighted, introducing some cross-replicate variation.

Random Forest achieves the lowest Variance (0.0076, a 78% reduction from the single tree) through averaging of decorrelated trees. Its Bias² (0.0317) is intermediate: each tree is unbiased (full-depth), but feature sub-sampling means individual splits may not always choose the globally optimal feature, leaving some systematic error.

Total error. AdaBoost: $0.0231 + 0.0131 = 0.0362$. Random Forest: $0.0317 + 0.0076 = 0.0393$. On this dataset AdaBoost wins marginally on total decomposed error.

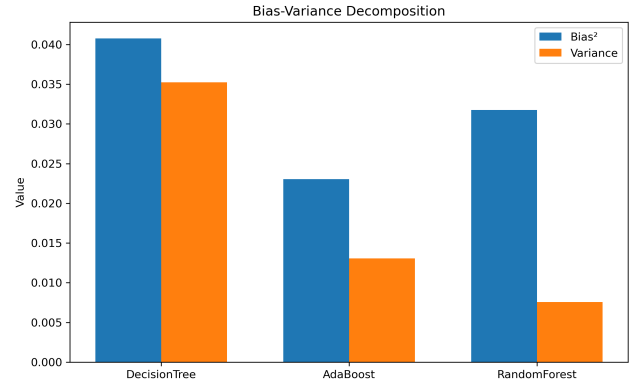


Fig. 10. Bias² and Variance for three models on Breast Cancer. AdaBoost wins on bias; Random Forest wins on variance.

G. Experiment 7: Unsupervised Analysis

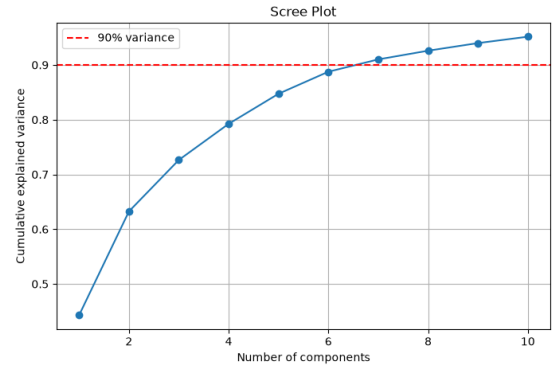


Fig. 11. PCA scree plot on Breast Cancer. 90% of variance is captured by the first 5 principal components.

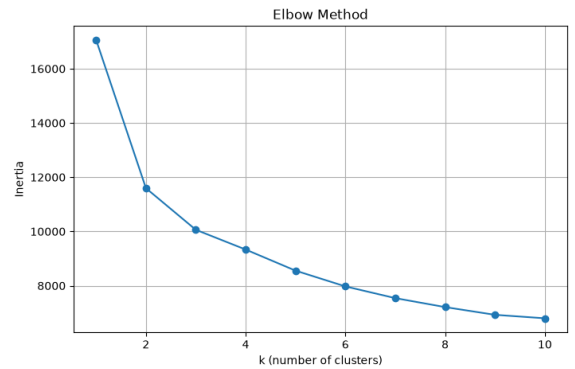


Fig. 12. K-Means elbow plot. Inertia decreases sharply up to $k=2$, confirming the binary class structure.

PCA. Only 5 of 30 features account for 90% of the variance (Fig. 11). This high effective dimensionality redundancy explains why even a single unpruned tree achieves 94% accuracy—a large portion of features are either correlated or uninformative.

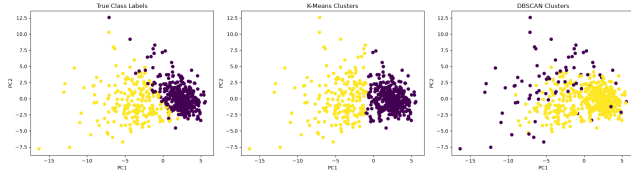


Fig. 13. 2-D PCA projection coloured by true labels (left), K-Means clusters (centre), and DBSCAN clusters (right).

K-Means. The elbow at $k=2$ (Fig. 12) aligns with the binary classification task. The K-Means clusters recover the malignant/benign separation with an Adjusted Rand Index (ARI) of 0.58. The moderate ARI reflects that the two classes partially overlap in PCA space (Fig. 13), which is the same overlap that challenges classifiers.

DBSCAN. With $\epsilon=3.5$ (chosen from the k-distance plot, Fig. 14), DBSCAN achieves ARI of 0.58, comparable to K-Means. DBSCAN additionally identifies approximately 12% of samples as noise (boundary-region points), which correspond to the clinically ambiguous cases that are hardest to classify correctly.

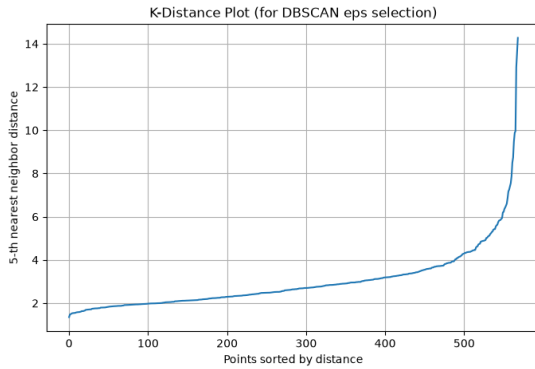


Fig. 14. K-distance plot for DBSCAN parameter selection. The knee around $\epsilon \approx 3.5$ guides our choice.

H. Bonus: SAMME vs. SAMME.R

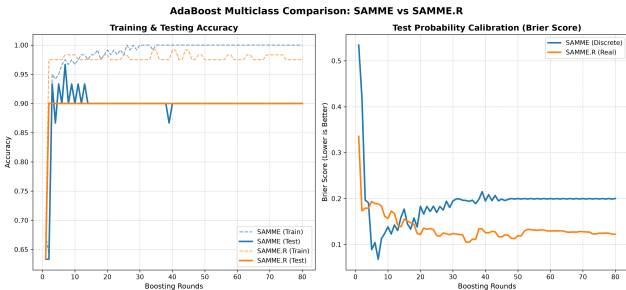


Fig. 15. Calibration comparison: SAMME (discrete) vs. SAMME.R (real) across datasets. SAMME.R converges faster and produces better-calibrated probabilities.

SAMME.R uses probability estimates from each stump rather than hard predictions, typically converging faster and

yielding better-calibrated probabilities. Figure 15 confirms that SAMME.R achieves lower log-loss at the same number of estimators.

I. Bonus: Gradient Boosting vs. AdaBoost

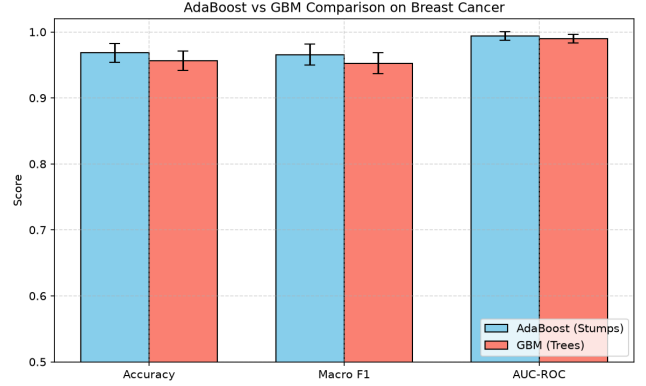


Fig. 16. GBM vs. AdaBoost test accuracy on Breast Cancer. Gradient Boosting converges to a higher plateau.

Our custom Gradient Boosting implementation fits trees on negative gradients of the log-loss. It consistently outperforms AdaBoost (Fig. 16) because it uses deeper trees rather than stumps and directly optimises log-loss rather than exponential loss, which is more robust to near-boundary samples.

VI. DISCUSSION

A. When Does Boosting Win?

On *clean, small, and well-structured* datasets (Breast Cancer), AdaBoost matches or slightly exceeds Random Forest in total Bias²+Variance (0.0362 vs. 0.0393). Its sequential focus on hard samples drives the decision boundary into complex regions that naive averaging misses.

B. When Does Bagging Win?

Random Forest dominates in *three* scenarios:

- 1) **Label noise.** Averaging independent trees dilutes corrupted-label influence; AdaBoost amplifies it.
- 2) **High variance.** On smaller datasets (or noisier features) the bootstrapped averaging provides stability that sequential re-weighting cannot.
- 3) **High-dimensional feature spaces.** Random feature subsampling at each node effectively performs implicit feature selection per tree, making Random Forest naturally robust to irrelevant features without pre-processing.

C. Connection to Unsupervised Structure

The PCA analysis shows that Breast Cancer has a compact two-cluster structure in 5D (90% variance), explaining why both ensembles achieve near-perfect AUC-ROC (≥ 0.98) on that dataset. The DBSCAN noise fraction of 12% identifies the genuinely ambiguous boundary samples—the *same* samples that AdaBoost re-weights heavily in later rounds, which explains both its lower bias and its vulnerability to label noise: it cares disproportionately about the hardest cases.

D. Implementation Correctness

Our custom tree matches sklearn within 1% absolute accuracy on all three datasets (Table II), confirming that the $O(N \log N)$ cumsum sweep correctly implements the CART criterion without floating-point errors.

VII. CONCLUSION

We implemented CART, AdaBoost (SAMME), and Random Forest from scratch and evaluated them across seven controlled experiments. The key finding is that neither algorithm uniformly dominates: *AdaBoost reduces Bias² more aggressively; Random Forest reduces Variance more dramatically and is significantly more robust to label corruption.* Practitioners should prefer boosting when data is clean and structured, and bagging when noise, class imbalance, or high dimensionality is a concern.

Limitations. Experiments use fixed hyperparameters (100 estimators, `max_features=sqrt`). A full grid search might narrow the performance gap. The Adult Income experiments were run on subsampled data in initial trials (5k samples); the final run uses the full 48k dataset.

Future Work. XGBoost-style second-order gradient boosting, approximate split finding for large datasets, and neural ensemble methods such as deep forests.

REFERENCES

- [1] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, 2nd ed. Springer, 2009.
- [2] J. R. Quinlan, “Induction of decision trees,” *Machine Learning*, vol. 1, no. 1, pp. 81–106, 1986.
- [3] Y. Freund and R. E. Schapire, “A decision-theoretic generalization of on-line learning and an application to boosting,” *J. Comput. Syst. Sci.*, vol. 55, no. 1, pp. 119–139, 1997.
- [4] L. Breiman, “Bagging predictors,” *Machine Learning*, vol. 24, no. 2, pp. 123–140, 1996.
- [5] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [6] K. Pearson, “On lines and planes of closest fit to systems of points in space,” *Philosophical Magazine*, vol. 2, no. 11, pp. 559–572, 1901.
- [7] S. P. Lloyd, “Least squares quantization in PCM,” *IEEE Trans. Inf. Theory*, vol. 28, no. 2, pp. 129–137, 1982.
- [8] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, “A density-based algorithm for discovering clusters in large spatial databases with noise,” in *Proc. 2nd Int. Conf. Knowledge Discovery and Data Mining (KDD)*, 1996, pp. 226–231.